Formal Modeling, Verification and Extended Implementation of a Secure ATM Transaction System using Automata Theory

Team: Trinabha Dixit (24BYB0026) · Bhargav Mahanta (24BYB0046)

THEORY OF COMPUTATION · PROJECT PRESENTATION

Aim & Objectives

This project bridges theoretical computation and real-world system design by modeling a complete ATM transaction lifecycle through the formal machinery of automata theory. Each computational model — from DFA to Turing Machine — plays a distinct role in capturing, validating, and securing transaction behavior.

Aim

Model and simulate ATM transactions using Theory of Computation concepts, applying formal computational models to a realistic, security-aware banking system.

Formal Validation

Validate transaction sequences rigorously using automata acceptance criteria, ensuring only well-formed input strings are processed through the system.

Model Integration

Integrate DFA, NFA, CFG, PDA, and Turing Machine — each addressing a distinct layer of transaction complexity, from input recognition to balance verification.

Secure Architecture

Build a structured, secure ATM system with deterministic flow, error handling, and built-in security mechanisms such as account lockout and session timeout.

System Overview

The ATM system follows a strictly deterministic state machine, progressing through well-defined stages from idle to transaction completion. Each stage corresponds to a formal automaton state, ensuring predictable and verifiable behavior throughout the user session.

Idle (q0) **Authentication** **Menu** **Processing** **Cash Dispense**

The pipeline above maps each user-facing stage to a corresponding automaton state, enabling formal verification of every valid and invalid transaction path.

Deterministic Flow

Every input maps to exactly one next state — no ambiguity in transaction routing or user interaction handling.

Invalid Input Handling

Malformed or unexpected inputs are captured by trap and dead states, preventing undefined system behavior.

Security via Lock & Timeout

Failed PIN attempts trigger `q_lock`; inactivity triggers session reset back to `q0`.

Real-Time Visualization

State transitions are rendered live using `vis-network`, making automata behavior observable during execution.

Module 1 — DFA-Based ATM Simulation

Formal Definition

A Deterministic Finite Automaton is defined as the 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where `δ(q, a)` maps each state-input pair to *exactly one* next state — the mathematical foundation of deterministic transaction routing.

State Inventory

01

q_0 — Idle

Awaiting card insertion; initial state of the automaton.

02

q_1–q_3 — Auth & Menu

PIN entry, verification, and operation selection stages.

03

q_4 — Accept State

Successful transaction completion; cash dispensed.

04

q_lock / q_dead

Trap states for account lockout and invalid sequences.

Modules 2 & 3 — NFA, ε-NFA & Context-Free Grammar

Non-Deterministic Finite Automaton

The NFA models scenarios where a single input may lead to **multiple possible outcomes** — such as PIN retry logic, where the system must account for success, failure, retry, or lockout simultaneously.

- **ε-transitions** enable internal state moves without consuming input
- Multiple outcomes for PIN: retry, success, lock, or failure
- Regular Expression: `(card)(pin)(op)(withdraw)(exit)`

Context-Free Grammar (CFG)

The CFG captures the **hierarchical structure** of a valid ATM transaction as a nested derivation, enforcing correct sequencing of all required operations.

S → card A

A → pin B

B → op C

C → withdraw D

D → exit

This grammar generates precisely the language of valid ATM transaction strings and rejects all malformed sequences.

Modules 4 & 5 — PDA & Turing Machine

The most computationally powerful models in the system handle stateful tracking and balance verification — tasks that finite automata alone cannot perform.

Pushdown Automaton (PDA)

The PDA augments finite control with a **stack memory**, enabling it to track nested transaction stages and maintain context across multi-step operations.

- **Push** operation records entry into a transaction stage
- **Pop** operation confirms stage completion
- Stack depth reflects transaction complexity

Turing Machine (TM)

The TM provides **unbounded tape-based computation**, enabling full verification of account balance against withdrawal amount — a task requiring arbitrary-precision arithmetic beyond PDA capability.

- Reads and writes symbols on an infinite tape
- Compares balance vs. withdrawal amount formally
- Outputs **Accept** (sufficient funds) or **Reject** (insufficient)

Security Mechanisms & System Features

Account Lockout

Three consecutive incorrect PIN entries transition the system to `q_lock`, preventing brute-force attacks on user credentials.

Dead State Trap

Any invalid input sequence routes execution to `q_dead`, a non-accepting trap state that halts further processing safely.

Session Timeout

Inactivity triggers an automatic reset to q_0, clearing session state and preventing unauthorized access after user departure.

Hardware Reset

A hardware-level recovery mechanism ensures the system can be forcibly returned to a known-safe initial state in emergency scenarios.

Additional Features: Real-time state tracking via `vis-network` visualization, a structured logging system for audit trails, and layered input validation at every automaton transition.

Working Test Cases

The system was validated against five canonical scenarios covering the full spectrum of ATM behavior — from successful transactions to security-triggered lockouts. Each case maps directly to a formal automaton state outcome.

✅ Successful Transaction

Valid card → correct PIN → valid operation → withdrawal → exit. System reaches accept state q_4.

🔁 Incorrect PIN (Retry)

Wrong PIN entered (fewer than 3 attempts). System allows retry, remaining in authentication loop without locking.

🔒 Account Lock

Three consecutive incorrect PIN entries. System transitions to `q_lock` and blocks further authentication attempts.

❌ Invalid Sequence

Out-of-order or malformed input (e.g., withdrawal before PIN). System routes to `q_dead` trap state.

⏳ Session Timeout

User inactivity exceeds threshold. System resets all state variables and returns to q_0 (Idle).

Formal Model & Implementation Architecture

Mathematical Model

The complete automaton is defined as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

With accepted language:

$$L = \{\text{card} \rightarrow \text{pin} \rightarrow \text{op} \rightarrow \text{withdraw} \rightarrow \text{exit}\}$$

All valid transaction strings belong to L; any string outside L is rejected by the automaton.

Three-Layer Architecture

Logic Layer — JavaScript

Core automaton engine implementing δ(q, a) transitions, state management, and all security rules.

UI Layer — HTML & CSS

Responsive interface for card insertion, PIN entry, menu selection, and transaction feedback.

Visualization — vis-network

Live graph rendering of automaton states and transitions, updated in real time as the user interacts.

Conclusion & Future Scope

Key Advantages

Real-World TOC Application

Demonstrates how abstract automata theory directly models production-grade system behavior.

Conceptual Clarity

Each computational model (DFA → TM) is isolated and validated, reinforcing theoretical understanding.

Viva & Demo Ready

Interactive visualization and formal proofs make this an ideal academic demonstration project.

Future Scope

Backend Integration

Connect to a real database for live balance and account verification.

Expanded ATM Features

Add fund transfer, mini-statements, and multi-account support.

AI Fraud Detection

Layer a probabilistic model atop the TM for anomaly-based fraud flagging.

Project Links

🔗 GitHub: github.com/ItsTrinabha/secure-atm-automata

🌐 Live Demo: *https://tocproject-one.vercel.app/*